

# **LA PROGRAMMATION MODULAIRE**

## **Introduction**

Généralement un programme est composé de plusieurs parties. Chaque partie symbolise un module. Pour des besoins de lisibilité, d'optimisation, de clarté et de réutilisabilité des codes, il est recommandé de créer pour chaque partie et de la considérer comme un module. Ces modules peuvent être utilisés dans des programmes. Pour cela ils doivent être appelés. On parlera d'appel de modules. Lors d'un appel de module l'appelant (émetteur) et l'appelé (récepteur) vont échanger à travers leurs arguments. Ces appels peuvent se faire en utilisant les modes de transmission. Le mode de transmission à utiliser est déterminé par le type de paramètre des arguments de l'appelé.

Chaque module est appelé un sous-programme et il peut symboliser une procédure ou une fonction. Dans un programme les variables seront classées en 2 catégories:

- les variables globales: elles sont déclarées dans un programme principal et elles peuvent être utilisées dans tout le programme
- Les variables locales : elles sont déclarées dans un module et elles ne sont utilisables que dans ces modules. Les variables locales sont composées des variables d'entête et intermédiaires.

## **I. La procédure**

### **1. Définition**

Une procédure est un module qui peut recevoir des arguments en donnée mais elle peut également renvoyer en résultats.

### **2. Prototype de la procédure**

```
PROCEDURE Nom_Procedure(Type_de_Parametre (s) Argument(s):Type_de_valeur(s))
// variable(s) intermédiaire(s)
Debut
    //Corps de la procédure
Fin
```

**Remarque :**

Donnée (D) = Entrée (E) = Input (I)

Résultat (R) = Sortie (S) = Output (O)

Donnée/Résultat (D/R) = Entrée/Sortie(E/S)=Input/Output (I/O)

**Dans quel cas il faut utiliser Donnée :** Le type de paramètre Donnée est utilisé si l'argument est indispensable à l'exécution du module

**Dans quel cas il faut utiliser Résultat :** Le type de paramètre Résultat est utilisé si l'argument est créé après l'exécution du module

**Dans quel cas il faut utiliser Donnée/Résultat :** Le type de paramètre Donnée/Résultat est utilisé si l'argument est indispensable à l'exécution du module mais sa valeur peut être modifiée pendant l'exécution du module

**Exercice d'Application 1 :**

Écrire une procédure qui permet de recevoir 2 entiers puis détermine leur somme

```

Procédure Somme(Donnees A,B:entier
                Resultat S:entier)

Debut
S <- A + B
Fin
  
```

**Exercice d'Application 2 :**

Écrire une procédure qui reçoit 2 entiers puis détermine leur différence.

```

Procédure Soustraction (Donnees A,B:entier
                        Resultat D:entier)

Debut
D <- A - B
  
```

Fin

### **Exercice d'application 3 :**

Soit un tableau d'entiers de 150 cellules, écrire une procédure qui détermine la moyenne des nombres pairs du tableau ainsi que le minimum et le maximum du tableau.

```

Const N=150
Type Tab = Tableau[1..N] Entier
Var T:Tab
Procédure Exercice3(Donnees T:Tab, N:Entier
                    Resultats moypair:Reel, mini,maxi:Entier)
var sompair,nbpair,i:entier
Debut
    sompair <- 0
    nbpair <- 0
    mini <- T[1]
    maxi <- T[1]
    Pour i allant de 1 a N Faire
        Si (T[i] mod 2 = 0) Alors
            sompair <- sompair + T[i]
            nbpair <- nbpair + 1
        FinSi
        Si (T[i]>maxi ) Alors
            maxi <- T[i]
        FinSi
        Si (T[i]<mini) Alors
            mini <- T[i]
        FinSi
    Finpour
    Si (nbpair != 0) Alors
        moypair <- sompair / nbpair

```

```

    sinon
        moypair <- 0
    FinSi
Fin

```

## II. La fonction

### 1. Définition

Une fonction est un module qui peut recevoir des arguments en donnée mais elle a une obligation de retourner toujours un résultat.

### 2. Prototype de la fonction

```

FONCTION nomFonction(Donnee(s) Argument(s):Type(s)):TypeValeurRetour
// variable(s) intermédiaire(s)
Debut
    // corps de la fonction
    retourner (expression)
Fin

```

**NB** : Le type de l'expression retournée par la fonction doit toujours être compatible au type de retour de la valeur de la fonction.

### **APPLICATION** :

Écrire une fonction qui reçoit un entier puis détermine le résultat de son factoriel

$$F = N \times (N-1) \times (N-2) \times \dots \times 1$$

$$F = 1 \times 2 \times 3 \times \dots \times (N-1) \times N$$

Solution 1 : en utilisant le parcours croissant

```

Fonction Factoriel(Donnee N:Entier):Entier
var i, f : Entier
Debut
    f ← 1
    Pour i allant de 1 a N Faire
        f ← f * i
    FinPour
    retourner f
Fin

```

Exercice :

Ecrire une fonction qui reçoit un tableau d'entiers de 150 cellules puis détermine la moyenne des nombres pairs du tableau.

```

Const N=150
Type Tab = Tableau [1..N] Entier
var T:Tab
Fonction Moyenne_nombre_Pair (Donnees T:Tab, N:Entier):Reel
var somPair, nbPair, i:Entier
var moyPair:Reel
Debut
    somPair <- 0
    nbPair <- 0
    Pour i allant de 1 a N Faire
        Si ( T[i] mod 2 = 0 ) Alors
            somPair <- somPair + T[i]
            nbPair <- nbPair + 1
        FinSi
    FinPour
    Si ( nbPair = 0 ) Alors

```

```

        moyPair <- 0
    Sinon
        moyPair <- somPair / nbPair
    FinSi
    Retourner (moyPair)
Fin

```

### **III. La recursivite**

La recursivite est un concept généralement utilisé en Maths sup mais elle peut être implémentée en informatique, alors on parlera de modules récursifs.

Un module est récursif s'il s'auto - appelle c'est à dire s'auto - réfère lors de son exécution. L'auto appellation peut se faire de façon directe ou transitive.

#### **Exemples de formules récursives**

##### **Exemple 1:**

$$N! = N \times (N-1)!$$

$$0! = 1! = 1$$

##### **Exemple 2:** Suite de Fibonacci

$$U_n = U_{n-1} + U_{n-2}$$

$$U_1 = U_2 = 1$$

##### **Application :**

Créer un module récursif qui reçoit un entier puis détermine son factoriel.

$$0! = 1! = 1$$

$$N! = N \times (N-1)!$$

```

Fonction facto_Recursive(Donnee N:Entier):Entier
Debut
    Si (N=0 ou N=1) Alors
        retourner 1

```

```
        Sinon
            retourner (N * facto_Recursive (N-1))
        FinSi
    Fin
```

```
Programme factoriel
Var N, F : entier

{
    N ← 5
    F ← facto_Recursive(N)
    Afficher ("Le factoriel de", N, " est", F);
}
```

**Remarque** : un module récursif n'utilise jamais de boucle. La répétition de ces traitements est liée à l'auto - appellation du module.